ESE 5370: Final Project Report

Parth Sarkar

Monday, April 20th, 2026

1 Introduction

Memory safety is a key priority for programs written in C. The burden of programing defensively against malicious use of undefined behavior lies squarely on the programmer, leading to mistakes that can be exploited by user input to the program. While the problem is pervasive and fundamentally a consequence of C semantics, the Softbound+CETS project offers the strong, formally-verified guarantee of *full* memory safety. Softbound+CETS instruments C code to create and update metadata for each pointer in the program at runtime. Upon a dereference for a pointer, the pointer's metadata is cross-referenced to ensure that the dereference occurs "within bounds" and is not a use-after-free.

However, this approach has two limitations. First, from a performance perspective, it is highly expensive to branch and check any condition on every dereference; [INSERT (revisited)] cites a 140% overhead on the SPEC CPU 2017 C benchmark suite. However, more pressingly, Softbound+CETS does not offer binary-level compatibility. Hence, if any C code that has not been compiled with Softbound+CETS instrumentation is linked into a protected project, the entire executable is susceptible to memory privilege escalation.

With these problems in mind, my final project first attempts to concretize the problem of privilege escalation due to foreign-linked code with a working attack that permits a buffer-overflow write (one that would normally be stopped by Softbound+CETS instrumentation if compiled with protection). As a proposed solution, I implement and compare two encryption methods on the metadata table that Softbound+CETS uses to propagate metadata on pointers stored into memory: a simple XOR encryption, and a more complicated AES encryption method. On the performance front, I implement and formally prove important properties of a global dataflow analysis in LLVM 12 and the Rocq interactive theorem prover, respectively. I integrate the results of this analysis into the Softbound+CETS pass and demonstrate correctness and tangible performance improvements. Ultimately, I hope my project makes the case that dynamic enforcement with strong security guarantees can go hand-in-hand with provably correct, classical static analysis methods, so the security-performance tradeoff can continue to be bridged.

2 The Attack

In order to motivate the encryption of the metadata table that is central to Softbound+CETS' memory safety guarantees, my project includes a simple attack embedded in an executable linked with one compilation unit unprotected by Softbound+CETS instrumentation. At a high level, given

knowledge about the location in virtual memory of the metadata table, the unprotected module can directly modify select entries of the table in order to escalate the privilege of a specific pointer. In my case, I have demonstrated this fact by modifying the table to allow, within the protected module, an offset from a stack allocated `Buffer A` to modify a disjoint section of the stack owned by `Buffer B`. While this is a simple example, I think it's clear that this is a limitation to the adoption of SoftBound+CETS instrumentation in legacy systems. Why should "protected" modules incur considerable overheads when there is no guarantee that data cannot be corrupted by pointers that live in and never even leave the protected module?

2.1 Relevant Background

First, in order to understand the way the attack was mounted, it is important to specify the mechamism that SoftBound+CETS uses to propagate and update metadata associated with each pointer.

2.1.1 Pointer & Metadata Origination

C semantics dictates that pointers can originate in multiple, legitimate ways. Most straightforwardly, the compiler can allocate variable, fixed-size buffers, and other statically-sized objects on the function's stack frame. Pointer variables that point to stack-allocated memory have a base, bound, and size that is determined at compile time. This metadata is inserted in-line with the pointer variable declaration.

In particular, here is my mental model of stack variable pointer transformations. With the following code:

```
struct student {
  int age;
  int id;
};

int main()
{
  struct student parth;
  parth.age = 23;
}
```

I expect to see LLVM intermediate representation code to the following effect:

```
...
%1 = alloca %struct.student, align 4
%1_age = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0

// Softbound+CETS inserted
%1_base = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 0
%1_bound = getelementptr inbounds %struct.student, ptr %1, i32 0, i32 2
%1_size = i32 4; // each field of 'student' holds 4 bytes
...
```

Here, the `getelementptr` (GEP) instruction is just computing a pointer offset from some base. Then, at the point where the variable `%1_age` is finally dereferenced, a branch is also inserted to verify the condition `%1_base <= %1_age + %1_size <= %1_bound`. In this example (where `fp` stands for "frame pointer"), the check simplifies to:

```
fp + 0 <= fp + 0 + 4 <= fp + 8
```

2.1.2 Pointer Propagation

An interesting case to deal with is when programmers need to store pointer variables themselves into memory (as a concrete example, the `task_struct` in the Linux kernel holds a pointer to the `pid` struct in its field `thread_pid`).

Here, SoftBound+CETS instrumentation is required to make a *metadata table entry* for the pointer value being stored to memory. The metadata cannot be reliably inlined because the pointer value will most likely escape the scope that any inlined metadata might belong to.

In particular, the metadata table maps each pointer stored into memory to a struct with `base` and `bound` values. The mapping uses a two-level table in order to balance quick look-up and modification times with reasonable memory usage. Specifically, the container I used to mount the attack used 48 bit pointers. The following diagram shows the way a pointer value is used to find its mapped metadata entry:

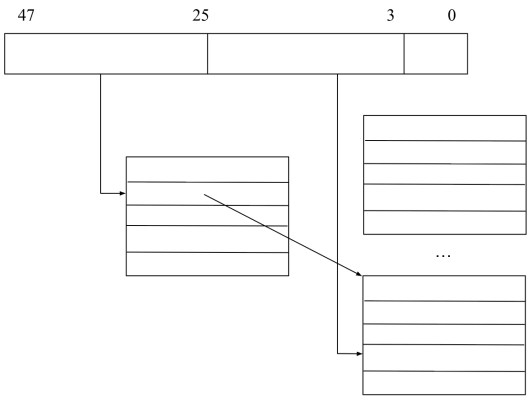

Figure 1: The Two-Level Lookup Table Structure

2.2 Attack Details

My attack employs an unprotected module to overwrite a key metadata table entry to escalate the privilege afforded to a pointer that lives in the SoftBound+CETS instrumented subset of the executable. This is not the only way to modify a pointer's metadata. For example, even the "inlined" metadata presented in 2.1.1 will exist in the memory if `main` calls a subroutine. A malicious subroutine might write beyond its stack frame, and modify the stack location pertaining to the `%1_bound` variable in its parent's stack frame.

However, my attack exploits the metadata table because I wanted to motivate the encryption of the metadata table as an initial, low-complexity encryption target (and get a rough understanding of overhead). It is possible (and necessary, for full protection) to encrypt metadata that lives on the stack, but that was a more involved process that will be left to future work.

In my attack, I initialize two adjacent arrays, `array1` and `array2`, and I store the pointer to `array1`'s first element is stored to memory.

```
/* main.c (protected) */

// these arrays live next to each other on the stack
int array1[] = {1, 2, 3, 4};
int array2[] = {5, 6, 7, 8};

// ensure pointer 'array1' has an entry in the metadata table by
// storing the pointer to a location in memory (i.e. not just inlined)
int *array1_ptr[] = {array1};
```

Next, in my unprotected module, having turned off ASLR, I hard-coded the address of the first-level metadata table. I then descended the two-level table, found the entry, and widened my privilege by 4 bytes in either direction.

In my protected module, I then used my escalated privilege to write beyond the bounds of `array1`, and influence `array2`.

```
/* main.c (protected) */


int *array1_alias = array1_ptr[0];
*(array1_alias - 1) = RANDOM_INTEGER;
*(array1_alias + 4) = RANDOM_INTEGER;
```

We can see the change from the stores above in the image below (`RANDOM_INTEGER = 37`). The directory with the code can be accessed here:
`https://github.com/parthsarkar17/softboundcets-ese5370/tree/main/attack`.

3 Encryption

As previously mentioned, my goal with encrypting the table was to prevent illegal modification of any table-stored metadata entries, which was the mechanism I used to mount the attack detailed above. Importantly, encryption does not prevent denial of service; an attacker might overwrite legitimate, encrypted data with nonsense bytes that halt the program when later decrypted and used. However, the main goal was to prevent the targeted escalation of privilege I demonstrated above.

Based on the two-level metadata table structure introduced in 2.1.2, there are several candidates for encryption. We can encrypt the contents of the first-level table (i.e. pointers to the second-level tables) or the contents of the second-level tables (i.e. the metadata themselves). In this project, I

elect to only encrypt the first-level table. I thought it was most important for a malicious program to not be able to access the metadata entry *at all*, compared to being able to access it and not be able to use or modify it.

Next, I will introduce and evaluate my two encryption methods on the metadata table.

3.1 Exclusive OR (XOR) Encrypted Table

The XOR (`^`) operator can be used as a basic encryption method. For a bit `b` and a one-bit key `k`, the operator has the property that `b ^ k ^ k = b`. Applied bitwise to a 64-bit value, a 64-bit key can provide cheap (i.e. one cycle ALU operation), symmetric encryption on sensitive data.

My XOR encryption scheme works as follows. First, I hard-coded a key `K` in the Softbound+CETS runtime library. On initialization of the first-level metadata table, I XOR each `NULL` pointer entry with `K`. Then, on every subsequent write and read from the table, I invoke functions that simply XOR the data with `K`. Since these functions only perform the XOR, I prompted the compiler to inline these calls. The key and functions I inserted are available here.

While I simply hard-coded `K` to some raw 64-bit value here, it is obviously important to generate a random value on each process initialization (in fact, Linux has a system call to generate a secure random number). Broadly, both my encryption efforts were focused on benchmarking performance, and the hard-coded value suffices here.

3.2 AES Encrypted Table

As a more serious form of protection, I then used AES to encrypt the pointers in the first-level metadata table. At a high level, AES is a symmetric encryption algorithm that encrypts a 16 byte block in 10 rounds, with a 128-bit key. The encryption procedure begins by expanding the key into 11 distinct 128-bit keys, one for each round and one additional. Then, each of the rounds applies an obfuscating function to the *state matrix* that initially encodes the plaintext. The resulting ciphertext is a highly secure, and it serves as the gold standard for my effort to encrypt the metadata table.

Because AES was more strict with its data format than the simple XOR above, I had to modify the first-level metadata table to store 16-byte blocks, instead of the 8-byte pointers it stored before. As such, I changed the first-level table from holding pointers to holding a fat pointer struct, with a size of 16-bytes.

Given more time, I would have liked to actually implement the AES encryption algorithm myself. Instead, I found a commonly-used, somewhat optimized C++ AES repository, and I modified it to link correctly with the Softbound+CETS C runtime library. My modifications to the runtime using the AES library are available here.

3.3 Results & Evaluation

I evaluate my implementation with respect to three metrics: memory overhead, concrete performance overhead, and security.

3.3.1 Memory Overhead

As an initialization step, Softbound+CETS sets up the first-level metadata table with `NULL` entries. As shown in Figure 1, the first-level table must be prepared to map bits `[47:25]` of a 48-bit pointer, for a total of 23 bits. Since each entry holds either an 8-byte `NULL` value or an 8-byte pointer to the corresponding second-level table, this first-level table occupies a total of $2^{23} \cdot 8 = 67$ MB by default. This overhead does not change for the XOR implementation. However, as mentioned in section 3.2, I needed to expand the each pointer entry in the first-level table to include 8-byte padding, in order to accomodate AES's 16-byte plaintext requirement. Therefore, my AES encryption method begins *every* process with a total of $2 \cdot 2^{23} \cdot 8 = 134$ MB of overhead by default.

3.3.2 Performance Overhead

While the overhead required by AES encryption over standard Softbound+CETS is large, I was expecting an even larger overhead when it came to performance.

The benchmark I used was the LINPACK linear system solver. This benchmark computes the LU factorization of a user-defined matrix A using Gaussian elimination, and then solves the linear system $Ax = b$ using this factorization. While my benchmark is not the diverse and varied set a real benchmark suite should aim to be, I did indeed discover a noticeable AES performance overhead for this benchmark. The following graph plots the raw time taken to factor and solve with an $n = 1000$ square matrix, for baseline `clang` (i.e. no Softbound+CETS), followed by my XOR and AES encrypted versions of Softbound+CETS. After a warmup period of 3 runs, I captured 5 executions of 16 iterations each, and took their average and standard deviation.

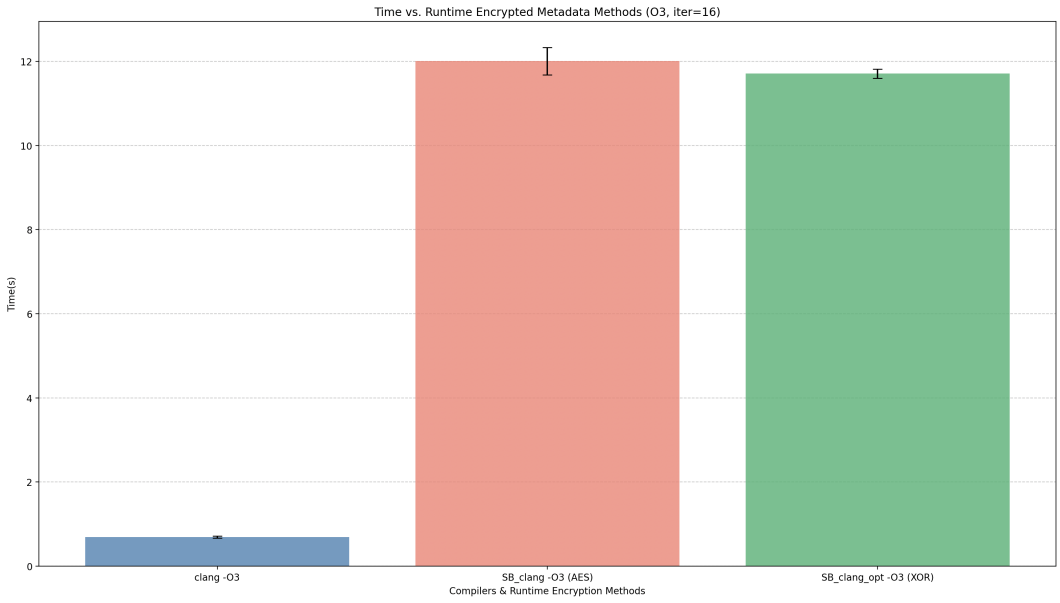

Figure 2: Time vs. Runtime Encrypted Metadata Methods (O3, iter=16)

The AES implementation took 12.006 seconds on average, the XOR implementation took an average of 11.708 seconds, and the unprotected `clang` execution took merely a 0.69 second average. For this benchmark, AES has only a 2.54% overhead relative to the XOR implementation. While this is exciting information, I have to acknowledge the following:

- First, I excluded the initial time it took to encrypt the first-level metadata table. Since this was an upfront, non-recurring cost, I considered the overhead to be sufficiently amortized over the course of the program.
- Second, this small overhead might be a limiation of the benchmark itself. It is possible that the number of pointers stored to memory in LINPACK is too small, and not representative of the "average" workload. Given more time, I would have liked to retry this experiment with a more representative suite.

3.3.3 Security Guarantees

The XOR and AES implementations both succeed in thwarting the attack I introduced in Section 2. My attempt to access the second-level table by indexing into the first-level table was stopped by each encryption mechanism, and resulted in a SegFault.

However, while the XOR implementation succeeded in halting the attack, has a much smaller memory footprint, and features encryption that requires only a one-cycle operation, it is also true that it offers quite weak encryption. For example, all an attacker must do to uncover the key is determine a mapping in the first-level table that *must* be `NULL (= 64'b0)`. Then, after observing the corresponding encrypted first-level table entry, it is trivial to deduce the key.

AES, however, is much more secure. As we have seen in class, its main attack vectors are side-channel (e.g. power) leakages to reveal keys at specific rounds. If given more time, I would have liked to extend AES encryption to cover stack-allocated metadata, thus providing complete metadata encryption for Softbound+CETS.